

Proyecto final AutoLife: Despliegue con Docker y Kubernetes

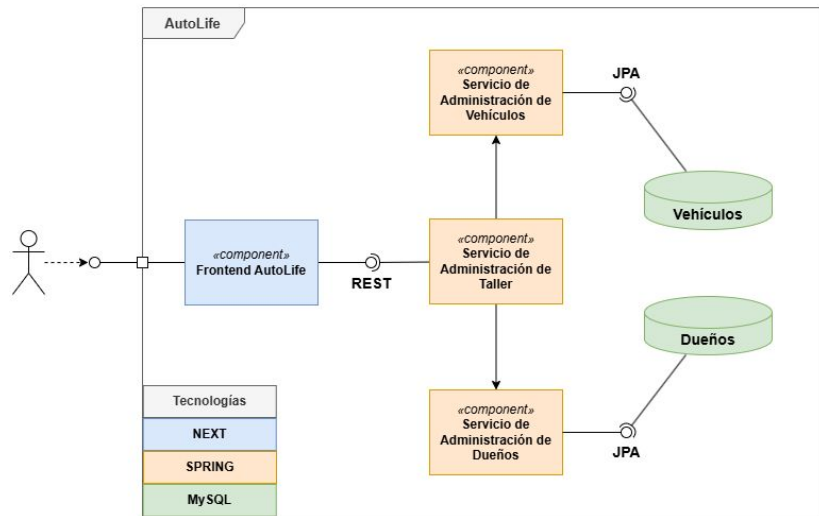
Integrantes:

Levir Heladio Hernandez Suarez - 2201776
Cristian Alexander Malagon Ruiz - 2200189
David Santiago Morantes Duarte - 2200178
Luis Andres Gonzalez Corzo - 2201493

Descripción general de la aplicación

La aplicación AutoLife tiene como propósito principal centralizar y automatizar la gestión de información en un entorno de talleres o servicios relacionados con vehículos. Está diseñada para manejar eficientemente los siguientes aspectos:

- **Gestión de Dueños:** Registrar, actualizar y eliminar información de propietarios de vehículos, asegurando un control preciso de los datos personales.
- **Administración de Vehículos:** Gestionar los vehículos asociados a cada dueño, incluyendo su registro, tipos y detalles relevantes.
- **Servicios del Taller:** Controlar la disponibilidad, actualización y registro de servicios ofrecidos por el taller.
- **Notificaciones Personalizadas:** Permitir la comunicación efectiva entre el taller y los usuarios mediante notificaciones segmentadas por rol.



Despliegue con Docker compose

```
GNU nano 7.2                                     docker-compose.yml
services:
  app:
    image: gradle:8.11.1-jdk21
    container_name: gradle-app
    working_dir: /app
    volumes:
      - /home/luis/docker/software3/AutoLifeBE:/app
      - gradle_cache:/home/gradle/.gradle
    command: >
      /bin/bash -c "
        ./gradlew build &&
        java -jar build/libs/*.jar"
    ports:
      - "8080:8080"
    depends_on:
      - mysql
    networks:
      - app-network

  mysql:
    image: mysql:8.0
    container_name: mysql-server
    environment:
      MYSQL_ROOT_PASSWORD: 1234
      MYSQL_DATABASE: AutoLife
    ports:
      - "3306:3306"
    volumes:
      - mysql_data:/var/lib/mysql
      - /home/luis/docker/software3/AutoLifeBE/src/main/resources/static/database.sql:/docker-entrypoint-initdb.d/database.sql
    networks:
      - app-network

volumes:
  gradle_cache:
  mysql_data:

networks:
  app-network:
```

Despliegue en Kubernetes: MYSQL DataBase

```
GNU nano 7.2
apiVersion: apps/v1
kind: Deployment
metadata:
  name: mysql
spec:
  replicas: 1
  selector:
    matchLabels:
      app: mysql
  template:
    metadata:
      labels:
        app: mysql
    spec:
      containers:
        - name: mysql
          image: mysql:8
          ports:
            - containerPort: 3306
          env:
            - name: MYSQL_ROOT_PASSWORD
              value: "1234"
            - name: MYSQL_DATABASE
              value: "AutoLife"
          volumeMounts:
            - name: mysql-storage
              mountPath: /var/lib/mysql
            - name: init-script
              mountPath: /docker-entrypoint-initdb.d/init.sql
              subPath: init.sql
      volumes:
        - name: mysql-storage
          persistentVolumeClaim:
            claimName: mysql-pvc
        - name: init-script
          configMap:
            name: mysql-init-script
```

```
---
apiVersion: v1
kind: Service
metadata:
  name: mysql
spec:
  ports:
    - port: 3306
      targetPort: 3306
  selector:
    app: mysql
```

```
GNU nano 7.2
apiVersion: v1
kind: PersistentVolume
metadata:
  name: mysql-pv
spec:
  capacity:
    storage: 2Gi
  accessModes:
    - ReadWriteOnce
  persistentVolumeReclaimPolicy: Retain
  hostPath:
    path: /var/lib/mysql
```

```
GNU nano 7.2
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: mysql-pvc
spec:
  accessModes:
    - ReadWriteOnce
  resources:
    requests:
      storage: 2Gi
```

Despliegue en Kubernetes: Aplicación Spring Boot

```
GNU nano 7.2
FROM eclipse-temurin:21
ARG JAR_FILE=autolife-0.0.1-SNAPSHOT.jar
COPY ${JAR_FILE} app.jar
ENTRYPOINT ["java", "-jar", "/app.jar"]
```

```
GNU nano 7.2
apiVersion: apps/v1
kind: Deployment
metadata:
  name: springboot-app
spec:
  replicas: 3
  selector:
    matchLabels:
      app: springboot
  template:
    metadata:
      labels:
        app: springboot
    spec:
      containers:
        - name: springboot
          image: luisagzc/spring:lastest
          ports:
            - containerPort: 8080
```

```
---
apiVersion: v1
kind: Service
metadata:
  name: springboot-service
spec:
  selector:
    app: springboot
  ports:
    - protocol: TCP
      port: 8080
      targetPort: 8080
  type: NodePort
```

Plan de pruebas

Se divide el plan de pruebas en 2 fases, la primera va relacionado a la variación de la cantidad de peticiones a lo largo del montaje en docker, micro k8s de 1 réplica, con 2 réplicas y con 3 réplicas. Para cada una de ellas se varió la cantidad de solicitudes de 50 a 9000 bajo las mismas condiciones para poder evidenciar el flujo de clientes que el backend puede soportar.

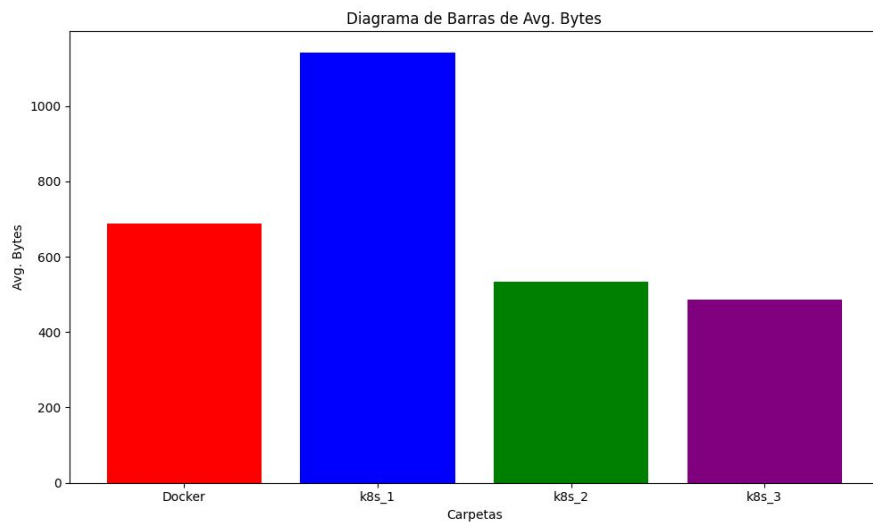
En la segunda fase, se realizan 50 solicitudes para cada uno pero variando esta vez el tiempo en milisegundos de cada uno de ellos, desde 1 hasta 1e-9 con el objetivo de evidenciar quién tiene un mejor tiempo de respuesta.

Métricas Consideradas

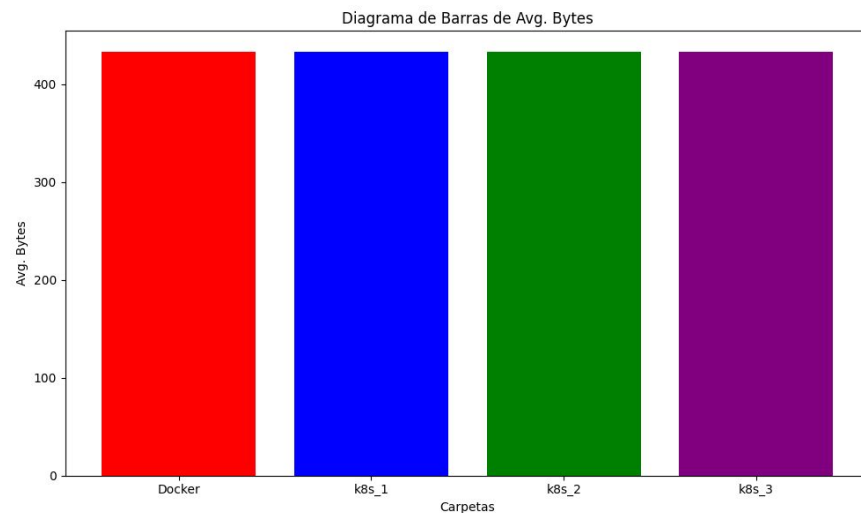
- ❖ Avg. Bytes: Tamaño promedio de las respuestas.
- ❖ Error %: Porcentaje de solicitudes fallidas.
- ❖ Min/Max: Tiempos más rápido y más lento registrados.
- ❖ KB/sec: Datos transferidos por segundo.
- ❖ Std. Dev.: Variabilidad en los tiempos de respuesta.
- ❖ Throughput: Solicitudes procesadas por segundo.

Resultados de las pruebas: Bytes promedio.

Solicitudes

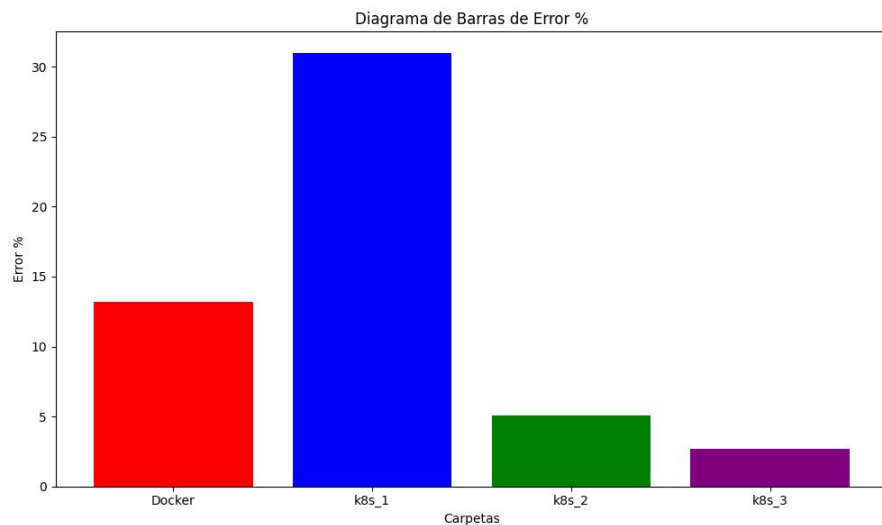


Tiempo de respuesta

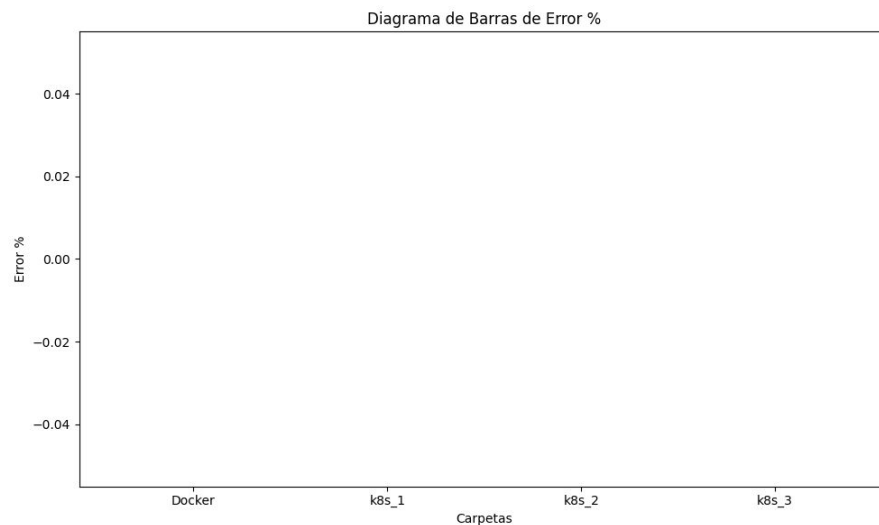


Resultados de las pruebas: Porcentaje de error.

Solicitudes

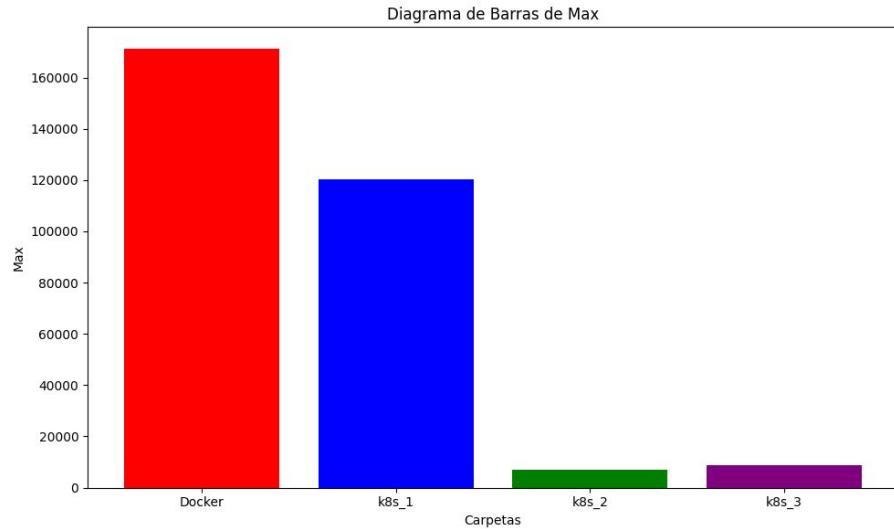


Tiempo de respuesta

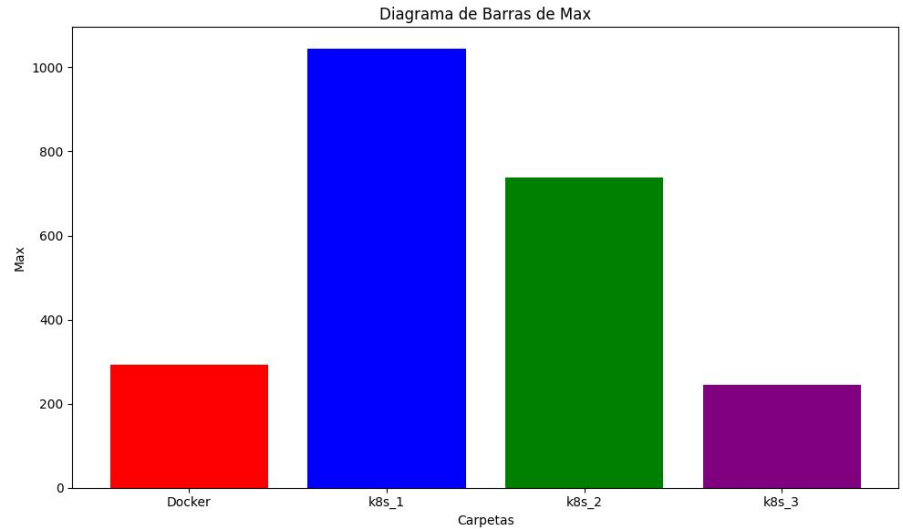


Resultados de las pruebas: Máximas.

Solicitudes

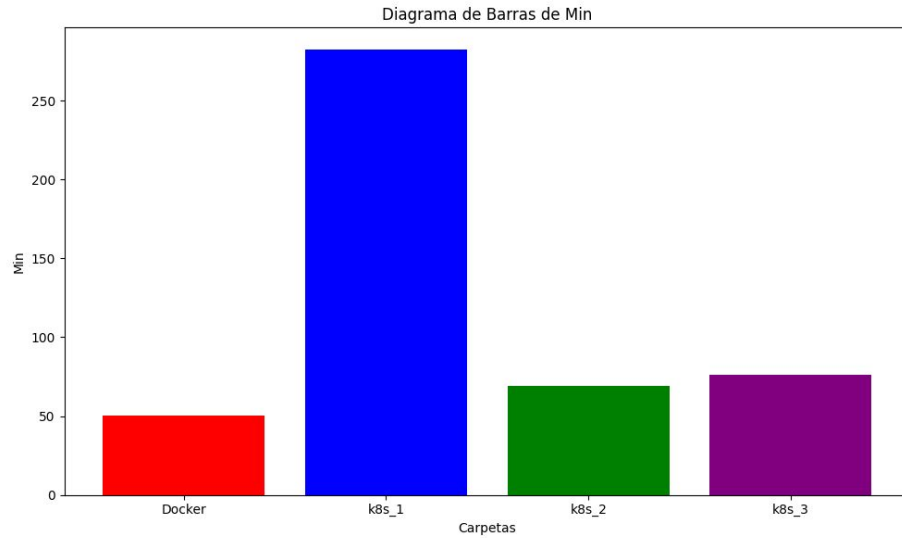


Tiempo de respuesta

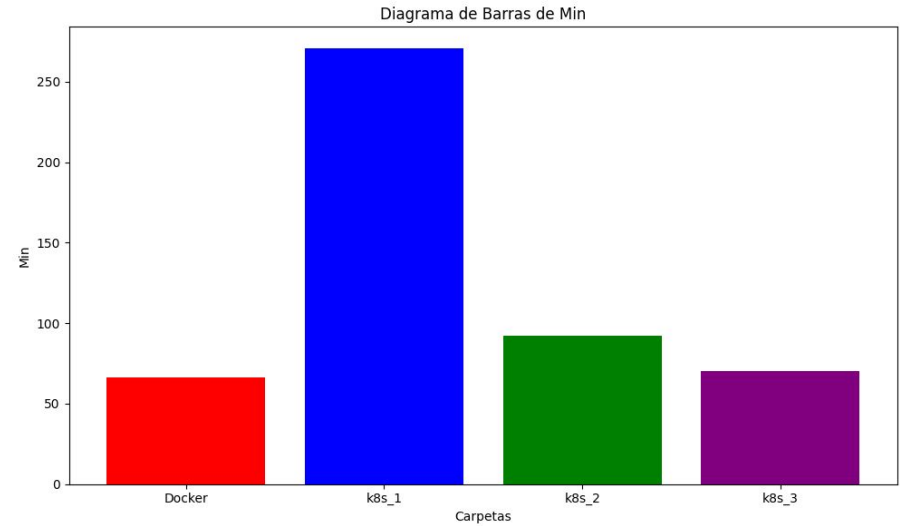


Resultados de las pruebas: Mínimas.

Solicitudes

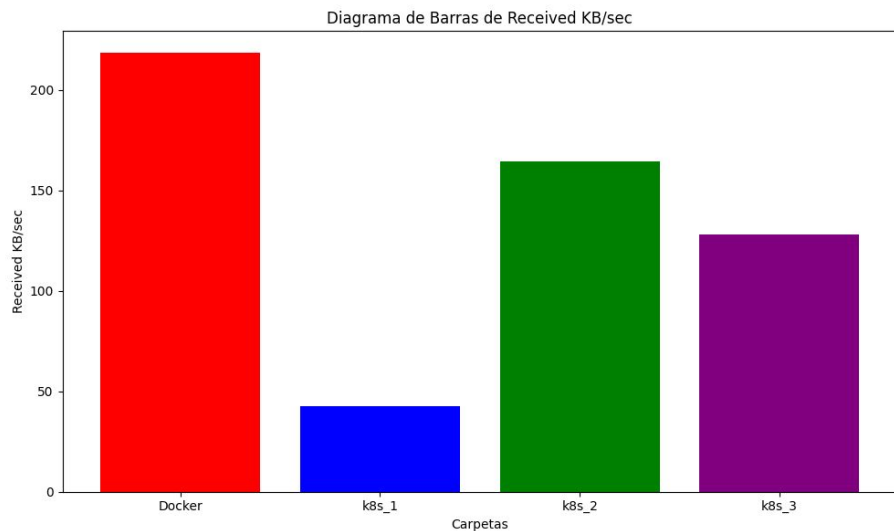


Tiempo de respuesta

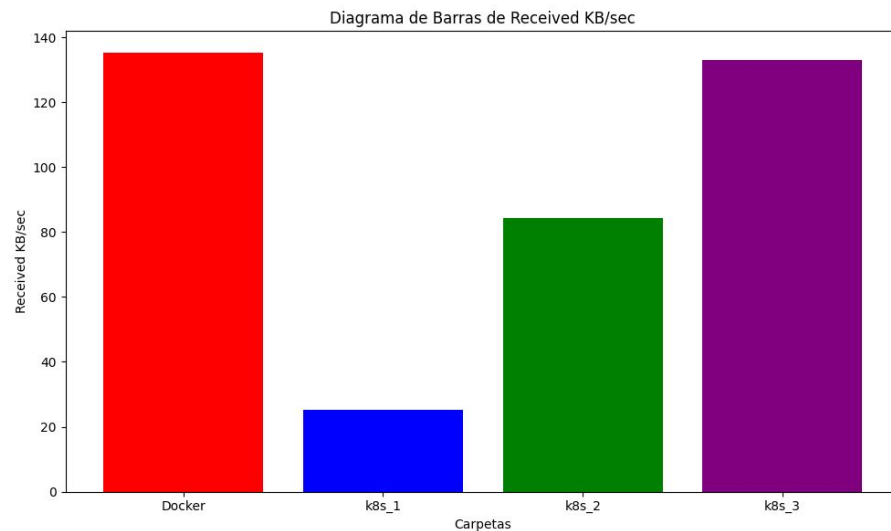


Resultados de las pruebas: KB recibidos por segundo.

Solicitudes

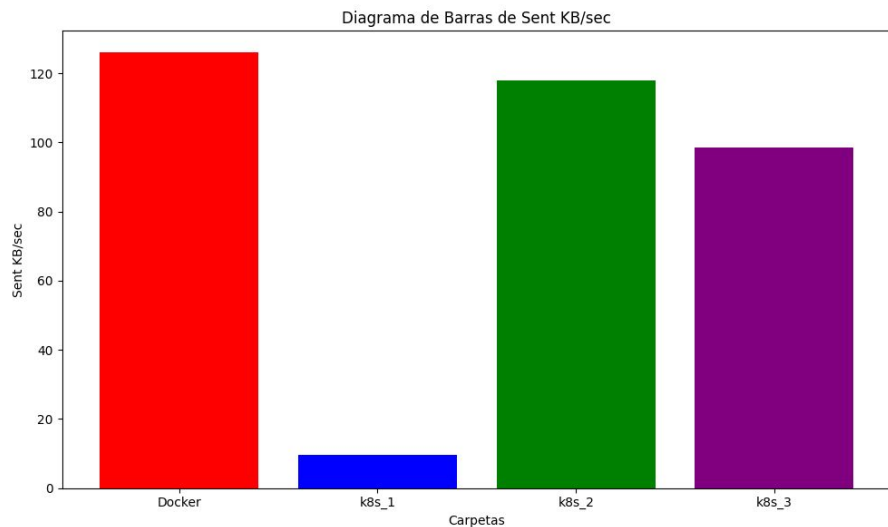


Tiempo de respuesta

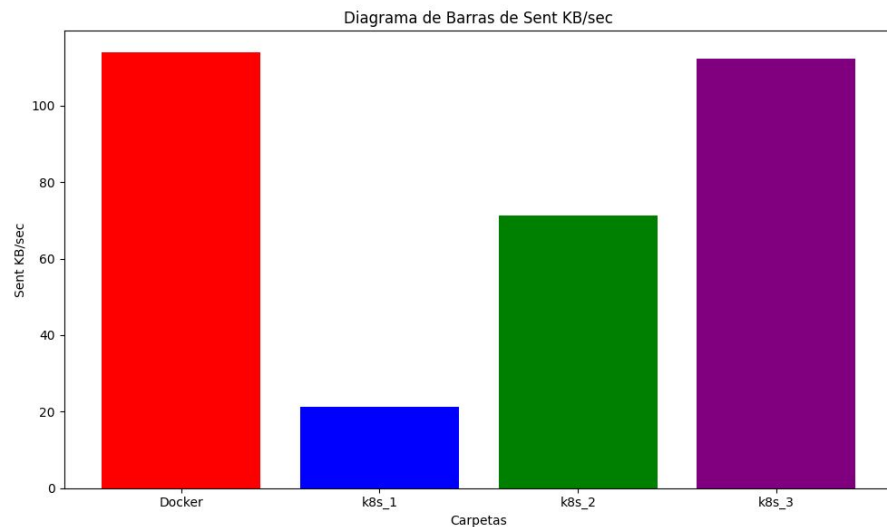


Resultados de las pruebas: KB enviados por segundo.

Solicitudes

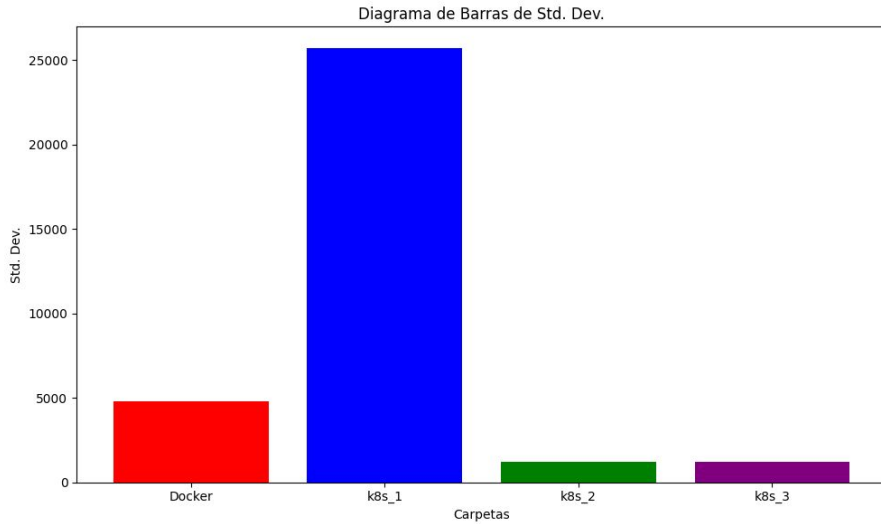


Tiempo de respuesta

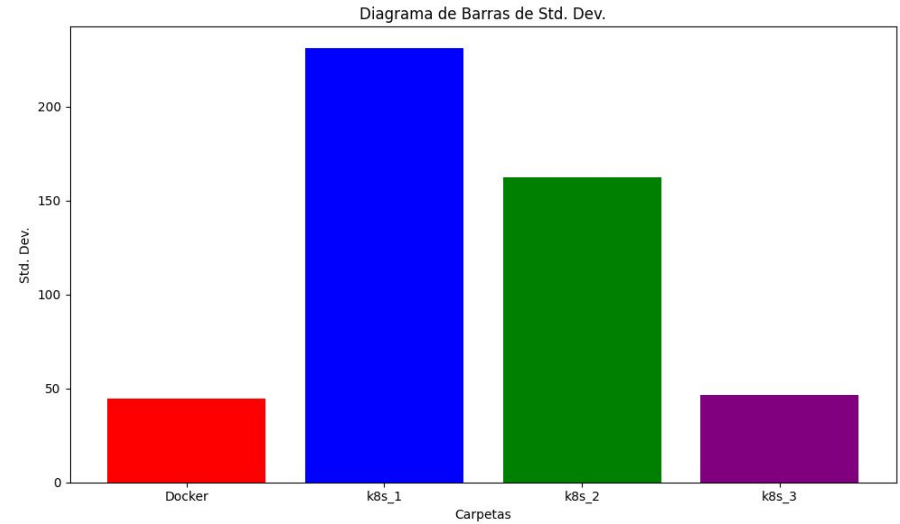


Resultados de las pruebas: Desviación estándar.

Solicitudes

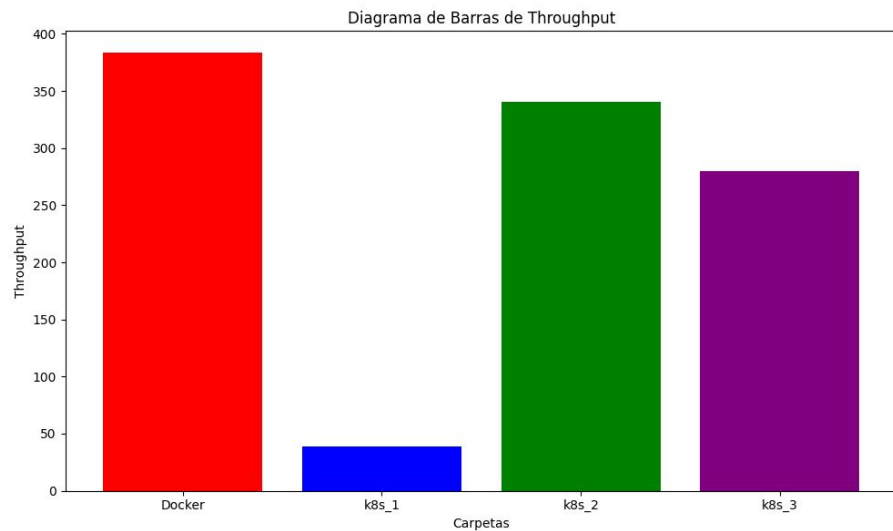


Tiempo de respuesta

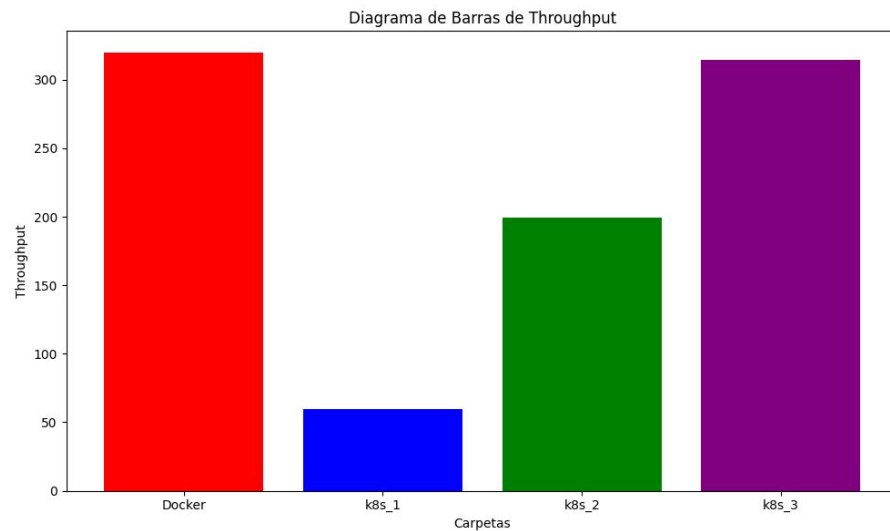


Resultados de las pruebas: Throughput.

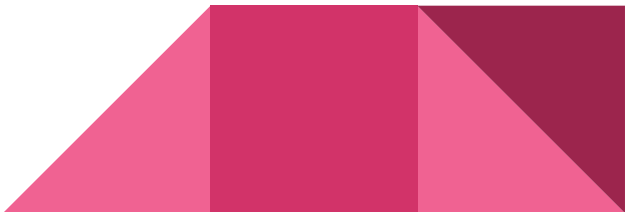
Solicitudes



Tiempo de respuesta



Conclusiones: Solicitudes

- MicroK8s con una réplica única presenta un rendimiento inferior al de Docker, debido a la sobrecarga de orquestación de Kubernetes, que consume más recursos y genera latencia adicional.
 - El uso de solo una réplica en MicroK8s crea un cuello de botella, ya que todas las solicitudes son manejadas por un único contenedor, sin distribución de carga ni redundancia.
 - MicroK8s, aunque más ligero que Kubernetes tradicional, sigue siendo más pesado que Docker en términos de consumo de recursos, afectando el rendimiento de las solicitudes.
- 

Conclusiones: Tiempo de respuesta

- Kubernetes y Docker muestran tiempos de respuesta similares, pero MicroK8s con una réplica única tiene un rendimiento inferior debido a la falta de escalabilidad y la gestión más compleja de pods y servicios.
- La asignación de recursos en MicroK8s no está tan optimizada como en Docker, lo que sobrecarga los recursos de un solo nodo, mientras que Docker maneja los recursos de manera más eficiente sin sobrecarga de orquestación.

[Informe del proyecto.](#)





¡GRACIAS
POR SU ATENCION!

